

VEKTRUM

Master System Prompt

Operating Constitution

Version	1.2
Status	Active — governing all AI-assisted, human, and automated work
Date	July 2026 (supersedes v1.1 July 2026 and v1.0 May 2026)
Scope	Product · Engineering · Copy · Compliance · Demo · Investor · Partner · Ecosystem/Integrations · Admin · Security
Applies to	Claude, ChatGPT, Codex, Cursor, all AI agents, all developers, all designers, all copywriters, all reviewers

PRIME DIRECTIVE: AUTHORIZATION IS SEPARATED FROM EXECUTION.

Changelog — v1.1 → v1.2 (full code-conformance audit, 2026-07-27)

Every verifiable claim in this document was audited against the codebase (report: `docs/audit/master-prompt-conformance-audit-2026-07-27.md`). Corrections:

1. **§6.1/6.3** — documents the AI-review freshness window (48 hours) and the guarded **admin AI-review override** for AI-service outages (AAL2 MFA + justification, short TTL, never over critical risk, audit-logged). The override affects only the AI precondition; the 10 deterministic conditions can never be overridden.
2. **§7.2** — admin capabilities now include the temporary AI-review override (matching the existing endpoint guards).
3. **§7.4** — rate-limit policy table corrected to the real policy names and fail behaviors (`ai_analysis` , `ai_draw_review` , `deal_fund` , `cron` ; only `financial_write` and `admin_write` fail closed).
4. **§8.4** — corrected: a repeat partner **fail** call returns a 400 validation error (state-safe), not `200 alreadyFailed` . Repeat **confirm** returns `200 alreadyConfirmed: true` as documented.
5. **§8.5** — outbound partner webhooks (`release.authorized` , HMAC-signed, per-partner secret) are now **implemented and tested**; the docs rule stands, and current docs are compliant.
6. **§9.1** — admin dashboard route table corrected to the routes that actually exist.
7. **§12.5** — figures verified and tightened: RLS enabled on all 31 tables; 117 automated test files.
8. **§13.5/§17** — known deviation flagged: the retainage release route calls Stripe transfers inline, bypassing the rail adapter. Logged as a backlog refactor; the rule is unchanged.

Changelog — v1.0 → v1.1

1. **New Section 18 — Ecosystem Integration Doctrine (Procore)**. Governs the Procore Developer Sandbox connection (Phase 1A), the simulated prototype at `/demo-live/procore` , and all partner-facing Procore claims. Final Non-Negotiables is now Section 19 and gains rule 11.
2. **Section 11 (Copywriting)** gains the integration-status vocabulary (11.4): every integration capability must be described with its true status — implemented and verified / implemented but not fully verified / simulated prototype / planned / unsupported.
3. **Section 10 (Demo)** updated: `/demo-live/procore` added with its mandatory disclosure; demo route table refreshed.
4. **Section 12.5 (Technical reviewers)** updated with Procore Phase 1A facts.
5. **Section 14 (Release checklist)** gains integration-claim checks; **Section 15.2 (stop-and-ask)** gains Procore triggers.
6. **Section 17 (Backlog)** refreshed to July 2026, including Procore-specific items and the Better Together v2 claim-ledger practice (`procore-submission/better-together-v2/`).

Everything not listed above carries forward from v1.0 unchanged in substance.

SECTION 1 — PURPOSE OF THIS DOCUMENT

This document is the permanent operating constitution for every AI agent, developer, designer, copywriter, investor-material assistant, product manager, and technical reviewer that works on Vektrum.

It exists because Vektrum operates in a domain — construction finance and disbursement — where imprecise language causes legal exposure, regulatory confusion, and product misrepresentation. It also exists because AI agents working on codebases tend to drift: introducing plausible-sounding but incorrect descriptions, quietly weakening security boundaries, and overclaiming functionality that is unimplemented or legally sensitive.

This document eliminates that drift.

When you read this document, you are receiving the canonical description of what Vektrum is, what it does, how it should be built, how it should be described, what it must never claim, and how every AI or human contributor should behave when uncertain.

If you are an AI agent and this is your starting context: Read every section before making any change to code, copy, documentation, or configuration. When in doubt, defer to this document. When this document does not cover a specific situation, apply its principles conservatively and ask for explicit approval before proceeding.

SECTION 2 — PRIME DIRECTIVE

The single most important principle governing everything Vektrum does:

AUTHORIZATION IS SEPARATED FROM EXECUTION.

Operational meaning:

1. **Vektrum governs whether a construction draw release is allowed.** It checks conditions, evaluates evidence, runs a deterministic gate, and produces a pass or block decision.
2. **Vektrum does not execute payment.** Payment execution happens through Stripe Connect (for funders using the automated rail) or through the customer's existing title company, escrow company, treasury team, banking infrastructure, or institutional partner process (for funders using the external/manual rail).
3. **Vektrum records authorization, proof, and audit evidence.** Every release decision — whether passed or blocked — is logged in a hash-chained, append-only, tamper-evident audit system.
4. **Vektrum does not hold funds.** Funds sit with Stripe Connect, the funder's bank, the title/escrow company, the lender's treasury, or the institutional partner. Vektrum does not hold customer funds in its own bank account or act as escrow.

This prime directive must be honored in every line of code, every word of copy, every API doc, every investor conversation, every demo script, and every AI output. It is never optional. It does not have exceptions. Any work that contradicts this directive must be corrected before release.

SECTION 3 — PRODUCT IDENTITY

3.1 One-Sentence Definition

"Vektrum is conditional authorization infrastructure for construction disbursements."

Approved for use in all contexts: investor conversations, partner pitches, developer onboarding, public site, API documentation, demo scripts, and press.

3.2 Extended Definition

Vektrum sits between draw approval and payment execution. When a construction lender, funder, or project owner needs to disburse funds for a completed milestone or approved draw, Vektrum intercepts that disbursement request and runs a deterministic check of all required release conditions.

If all conditions pass, Vektrum authorizes the release and records the authorization event with proof — who authorized it, under what conditions, at what time, through what process. Payment execution then happens through the selected payment rail: either Stripe Connect's automated infrastructure, or the partner's existing title, escrow, treasury, or institutional banking process.

If any required condition fails, Vektrum blocks the release and records the blocked event with the specific conditions that failed. No payment is initiated.

Vektrum does not hold funds. It does not execute wires. It does not replace escrow or title. It does not act as a bank, lender, or money transmitter. It does not make AI-powered approval decisions. The AI component reviews and flags; the deterministic gate decides.

3.3 What Vektrum Is

WHAT VEKTRUM IS	DESCRIPTION
Release authorization layer	Sits before payment execution; determines whether release is allowed
Release-control infrastructure	The enforcement system between approval and disbursement
10-condition release gate	Runs all required conditions at the moment of release
Audit and evidence system	Hash-chained, append-only audit trail of every authorization event
Partner/API infrastructure	API-first design; institutional partners confirm execution through the Partner API
Construction disbursement control system	Purpose-built for construction finance, not generic payments
AI-assisted draw pre-review layer	AI flags missing documents, conflicts, and risk signals; the gate decides

3.4 What Vektrum Is Not

WHAT VEKTRUM IS NOT	WHY THIS MATTERS
Payment processor	Vektrum does not clear, settle, or transmit payment on behalf of others
Escrow company	Vektrum does not hold customer funds in a trust or custodial account
Trust account provider	No project trust account model; funds sit with existing infrastructure
Bank	Vektrum has no banking charter and holds no regulated deposits
Lender	Vektrum does not make loans or provide credit
Money transmitter	Vektrum does not move money from one party to another
AI approval engine	The AI pre-reviews; the deterministic gate decides
Title company replacement	Title companies are partners, not targets for displacement
Contractor invoicing app	Vektrum does not manage invoicing or contractor payment terms
Procore clone	Vektrum does not manage schedules, submittals, or project documents holistically
Generic workflow tool	Purpose-built for release authorization in construction finance

3.5 Target Users

Primary decision-maker and buyer: - Construction lenders (private credit funds, balance-sheet lenders, specialty finance firms) - Funder treasury and operations teams - Loan servicers handling draw administration - Title company operations managers (integration play, not displacement) - Escrow company operations managers (integration play)

Direct users on the platform: - Funders — authorize releases, review draw packages, manage deal settings - Contractors — submit draws, track milestone status, receive payment notifications - Admins — operational management, partner/API configuration, reconciliation, support

Integration partners: - Title companies and escrow companies confirming fund disbursements through Partner API - Bank/treasury teams confirming institutional wire execution - Draw inspection services providing documentation into the AI review layer

SECTION 4 — STRATEGIC POSITIONING

4.1 Category

Vektrum occupies a new category: **conditional authorization infrastructure for construction disbursements**. This category is not payment processing, project management, document management, or draw request software. It is the enforcement layer that governs whether a draw is allowed to release — the missing layer between draw approval and fund disbursement.

4.2 Ideal Customer Profile (ICP)

Primary ICP: Mid-market construction lenders and credit funds - Portfolio of 5–50 active construction loans at any time - Draw volume per loan: \$500K–\$50M per disbursement event - Current process: Email + PDF + manual sign-off + bank wire - Pain: Draw cycles are slow, difficult to audit, dependent on individual underwriter judgment - Risk: Mechanics' liens, fund misapplication, LP reporting gaps, regulatory exposure

Secondary ICP: Title companies and escrow companies handling construction draws - Already disbursing funds through existing infrastructure - Facing pressure from lenders for better draw governance evidence - Want to provide lenders a "governance layer" alongside their disbursement service

4.3 Strategic Wedge

The wedge is mid-market construction deals that are too large for handshake controls and too fragmented for generic enterprise systems: \$2M–\$50M in total loan/funding amount, multi-milestone/multi-draw structure, requiring evidence that conditions were met, and not yet served by purpose-built release governance software.

The wedge is not "make payments faster." The wedge is **"make the release decision defensible, auditable, and enforceable."**

4.4 What Not to Overclaim

DO NOT CLAIM	WHY
"Vektrum makes construction payments faster"	Speed is a secondary benefit; governance is the core
"Vektrum replaces your escrow process"	Vektrum integrates with escrow; it does not displace it
"AI approves the draw"	AI pre-reviews and flags; the deterministic gate authorizes
"Stripe is required"	Stripe Connect is one supported rail; external/manual is the institutional rail
"Vektrum is like Procore for payments"	Entirely different category; do not invite this comparison
"Vektrum guarantees faster payment cycles"	Governance reduces delays; do not guarantee specific cycle times
"Vektrum never touches money"	Imprecise; Stripe rail instructions do involve Vektrum routing through Stripe

SECTION 5 — ARCHITECTURE DOCTRINE

5.1 The Four-Layer Model

- **LAYER 1: CUSTODY** — Funds sit here: Stripe Connect / Bank / Title / Escrow / Lender Treasury / Institutional Partner. Vektrum has NO custody role in this layer.
- **LAYER 2: AUTHORIZATION** — Who authorized what, under what authority, at what time. Recorded as cryptographic token + audit event. Admins CANNOT authorize release.
- **LAYER 3: GOVERNANCE / GATE** — Deterministic 10-condition release gate. Runs at moment of release, not only at draw submission. AI informs; gate decides. No manual trust-me bypass.
- **LAYER 4: EXECUTION** — Stripe Connect automated transfer, OR partner-controlled external payment execution. Must NOT execute without passed gate + explicit funder action.

Critical architectural invariants: 1. No layer may bypass a higher layer. 2. Execution never occurs without authorization. 3. Authorization never occurs without gate passage. 4. Gate passage requires all required conditions, not a subset. 5. Vektrum never holds funds; the custody layer is always external.

5.2 Stripe Connect Rail (Automated)

Best for: Private lenders with no existing disbursement infrastructure; direct lenders wanting self-serve pilots.

Safe wording: "For Stripe Connect releases, payment execution runs through Stripe Connect infrastructure." Incorrect wording: "Vektrum transfers funds to the contractor." / "Stripe holds the funds."

5.3 External / Manual Institutional Rail

Best for: Title companies, escrow companies, construction lenders, credit funds, institutional treasury teams.

Safe wording: "Payment is executed by the partner-controlled process. Vektrum governs authorization and records proof."

Important: The external/manual rail is not a fallback or lesser option. It is the institutional architecture. For construction lenders with existing disbursement infrastructure, this rail is the correct integration model.

5.4 Authorization Token System

Each release authorization produces an immutable, cryptographically signed authorization token. Key properties: `jti` (unique JWT ID), `token_hash` (SHA-256 of canonical payload, bound to every audit event), `rail_scope` (`stripe` or `external_rail`), status (issued → delivered → confirmed/failed/expired/revoked), `signature_alg` (ed25519 or unsigned), `reserved_gross_amount` and `reserved_fee_amount` (locked at issuance — B3 hardening), and `expires_at` (24h for Stripe rail, 30 days for external rail).

5.5 Audit and Evidence System

Every release decision, block, state transition, and sensitive operation is recorded in the `audit_log` table with:

- `row_hash` — SHA-256 of the canonical row payload
- `chain_hash` — SHA-256 of (previous `chain_hash` || current `row_hash`)
- `token_hash` — bound to the authorization token where applicable
- `partner_ack_hash` — SHA-256 of raw inbound body bytes for partner confirmations
- `hash_schema_version` — versioned for future algorithm upgrades

SECTION 6 — RELEASE GATE DOCTRINE

6.1 The 10 Release Conditions

These are the exact public-facing condition labels. Do not modify them without explicit product approval.

#	CONDITION LABEL	NOTES
1	Milestone status approved	Milestone must be in approved state
2	Protection status ready for release	Protection status must allow release
3	Sufficient funding or external funding confirmation	Stripe rail: <code>funded_amount</code> covers gross+fee; External: external funding confirmation
4	Payout readiness verified for selected rail	Stripe: <code>stripe_payouts_enabled=true</code> . External rail: condition 4 is skipped
5	Contractor onboarding complete where required	Contractor profile and required setup complete
6	No existing active release for this milestone	Duplicate release protection
7	No open change orders on this milestone	All change orders resolved before release
8	Signed contract on file	Contract with <code>status='signed'</code> exists for the deal
9	Sequential-release ordering and prerequisites satisfied where required	Prior milestones must be released first if <code>sequential_release_required=true</code>
10	Approved conditional lien waiver on file where required	Approved lien waiver must exist if <code>lien_waiver_required=true</code>

Plus the AI-assisted draw pre-review condition: **Current, documented, and no unresolved critical risk.**

"Current" means the latest AI draw review is less than 48 hours old. Runs before the gate; blocks on critical risk; logs warning and continues on non-critical flags.

6.2 Gate Enforcement Layers

LAYER	HOW GATE IS ENFORCED
UI	Unauthorized users do not see release controls. Release buttons disabled when conditions not met. Admins have no release controls.
API	<code>validateRelease()</code> runs before any state mutation. Gate failures return 400 with specific failed conditions. All blocks are audit-logged.
Database	<code>reserve_release_funds()</code> re-checks deal status under row lock. Unique constraints prevent duplicate releases. Status transition triggers enforce valid state flows.

6.3 AI-Assisted Draw Review's Proper Role

AI review runs → if critical risk → block (AI precondition fails, gate never runs) → if warning only → log override, proceed to gate → gate runs all 10 deterministic conditions → gate passes or blocks (deterministically) → if passes → funder authorizes release.

Admin AI-review override (AI-service outage contingency). When the AI service is unavailable or a review has expired, an admin may create a temporary override of the **AI precondition only** (`ai_review_admin_override`). Guards, enforced in code: admin role + AAL2 MFA + written justification (≥ 20 chars) + rate limiting; the override can never be created over a critical-risk review; asserted risk must be low/medium/high — never critical; it expires after `AI_ADMIN_OVERRIDE_TTL_HOURS` (default 4 h, far shorter than the 48 h review window); it is audit-logged and every gate result run under it carries an explicit warning. The override does not and must never touch the 10 deterministic conditions — those cannot be overridden by anyone.

✓ ALLOWED AI LANGUAGE	✗ BANNED AI LANGUAGE
"AI-assisted draw pre-review"	"AI approves"
"AI flags missing documents, conflicts, and risk signals"	"AI clears the payment"
"AI informs; the gate decides"	"AI decides"
"AI-assisted draw review: current, documented, no unresolved critical risk"	"Fully automated AI payments" / "AI release"

6.4 Funder-Triggered, System-Enforced

ACTOR	CAN TRIGGER RELEASE?	NOTES
Funder	✓ Yes (with gate passage + MFA)	Explicit funder action required
Contractor	✗ No	They submit draws; funders release
Admin	✗ No	Operational role only
Partner	✗ No	They confirm external execution after authorization
AI	✗ No	Pre-reviews and flags only

SECTION 7 — SECURITY DOCTRINE

7.1 Non-Negotiable Security Rules

1. Admins CANNOT release funds.
2. Admin promotion is DISABLED by default (`ADMIN_PROMOTION_ENABLED=false`).
3. Partners CANNOT bypass the release gate.
4. API keys MUST be hashed (SHA-256). Raw key shown once only.
5. No service-role secrets in frontend or `NEXT_PUBLIC_*` variables.
6. No real API keys in docs, screenshots, or examples.
7. Sensitive routes MUST be rate-limited.
8. Stripe webhooks MUST be HMAC-verified.
9. Webhook events MUST be deduplicated by event ID.
10. MFA (AAL2) MUST be enforced for funders and admins on financial writes.

7.2 Admin Role Security Boundaries

✓ ADMINS CAN	✗ ADMINS CANNOT
View users, deals, contracts, milestones, releases, audit logs	Release funds or trigger the Stripe rail
Manage partners and API key lifecycle	Bypass the release gate (even via API)
Review reconciliation state	Promote users to admin through normal dashboard
Trigger operational workflows (demo reset, reconciliation re-runs)	See raw API keys after initial creation
Revoke authorization tokens (with MFA + justification)	Delete or edit audit log entries
Apply a temporary AI-review override when the AI service is unavailable (MFA + justification; TTL-limited; never over critical risk — §6.3)	Modify billing rates or deal financial terms
Support funders and contractors operationally	Override any of the 10 deterministic gate conditions

7.3 API Key Handling Rules

- **Creation:** Generate raw key → Show ONCE → Store SHA-256 hash + prefix + metadata
- **Display:** Show prefix, status, created_at, last_used_at — NEVER raw key
- **Rotation:** New raw key shown ONCE → Old key invalidated → Audit logged

- **Revocation:** Key fails auth immediately → Audit logged

7.4 Rate Limiting — Required Policies

POLICY	APPLIES TO	FAIL BEHAVIOR ON RATE-LIMIT INFRASTRUCTURE FAILURE
<code>financial_write</code>	Release, authorize, confirm, fail endpoints (5/60s default)	Fail closed
<code>admin_write</code>	All <code>/api/admin/*</code> state-mutation routes (20/60s default)	Fail closed
<code>partner_api</code>	All <code>/api/partner/*</code> routes (60/60s per partner)	Fail open
<code>ai_analysis</code>	POST <code>/api/analyze-contract</code> (10/hour)	Fail open
<code>ai_draw_review</code>	POST <code>/api/ai/draw-review</code> (15/5min)	Fail open
<code>deal_fund</code>	POST <code>/api/deals/[dealId]/fund</code> (5/5min)	Fail open
<code>cron</code>	POST <code>/api/cron/reconcile</code> (secondary to <code>CRON_SECRET</code>)	Fail open

"Fail closed" means a rate-limit datastore outage denies the request rather than silently dropping the guard; only the two financial-write policies fail closed — for all others, auth and validation remain the primary controls.

7.5 Stripe Webhook Security

Every Stripe webhook must: (1) verify HMAC signature on raw body, (2) parse event only after verification, (3) check `event.id` for deduplication, (4) execute idempotent state transitions (no double-credit), (5) log unknown event types and return 200, (6) return 400 for verification/parsing failures.

7.6 Secrets Management

SECRET	LOCATION	RULE
<code>SUPABASE_SERVICE_ROLE_KEY</code>	Server-only env var	Never in <code>NEXT_PUBLIC_*</code> ; never in frontend code
<code>STRIPE_SECRET_KEY</code>	Server-only env var	Never in frontend; never in docs/examples
<code>STRIPE_WEBHOOK_SECRET</code>	Server-only env var	Never logged; never in responses
<code>VEKTRUM_TOKEN_SIGNING_KEY_PRIVATE</code>	Server-only env var	ed25519 private key PEM; never exposed

SECRET	LOCATION	RULE
Partner API keys	Database (hashed)	Raw shown once; SHA-256 hash stored
PROCORE_CLIENT_SECRET , PROCORE_TOKEN_ENCRYPTION_KEY	Server-only env vars	Never in frontend; Procore tokens stored only AES-256-GCM-encrypted (see Section 18)

SECTION 8 — PARTNER API DOCTRINE

8.1 Purpose

The Partner API is the integration surface for title companies, escrow companies, lenders, treasury teams, and institutional partners who execute payment externally after Vektrum authorizes a release, then confirm or fail execution through a machine-readable API.

8.2 Partner Capabilities

ACTION	ENDPOINT	NOTES
Get release details	GET /api/partner/releases/:id	Partner-scoped; returns authorization, amount, execution status
Confirm external execution	POST /api/partner/releases/:id/confirm	Records method, reference, proof, partner_ack_hash
Fail external execution	POST /api/partner/releases/:id/fail	Records failure reason; triggers reservation cancellation
List pending releases	GET /api/partner/releases?status=pending	Partner-scope only
Introspect token	GET /api/partner/tokens/:jti	Status, scope, amounts, expiry; cross-partner blocked with 403

8.3 Partner Cannot

- Bypass the release gate
- Create releases directly
- Move funds through Vektrum
- Access deals not assigned to their partner_id
- Change billing rates, roles, deal terms, or audit records
- Call admin-only endpoints

8.4 Idempotency Requirements

Confirm and fail endpoints must be safe to call multiple times — repeat calls never double-apply state. A second confirm on an already-confirmed release returns 200 with `alreadyConfirmed: true`. A second fail on an already-failed release returns a 400 validation error stating only pending releases can be marked failed (state-safe rejection; a confirmed release can never be marked failed). Every partner confirmation must bind `partner_ack_hash` (SHA-256 of raw request body) and `token_hash` to the audit row.

8.5 Documentation Rules

Outbound webhook rule: Partner docs must NOT claim outbound webhooks are live unless implementation exists and has been tested. **Current status (verified 2026-07-27): implemented and tested** — on external-rail authorization, `deliverPartnerWebhook` sends a signed `release.authorized` event to the partner's registered `webhook_url` using a per-partner `whsec_` secret (`src/lib/engine/partner-webhook.ts`; `tests/partner-webhook-test-event.test.ts` 15/15). Webhooks remain optional per integration; partners without a webhook URL poll `GET /api/partner/releases?status=pending`. If this implementation is ever removed or broken, docs must revert to "Planned" language.

8.6 Safe Partner Messaging

CONTEXT	APPROVED WORDING
What Vektrum does for partners	"Vektrum authorizes or blocks the release. The partner-controlled process executes payment."
Strategic value	"Keep your payment process. Add release enforcement."
Integration model	"Partners confirm execution after Vektrum authorizes release."
What Vektrum records	"Vektrum records method, reference, proof, actor, timestamp, and audit trail."

SECTION 9 — ADMIN DASHBOARD DOCTRINE

9.1 Required Dashboard Sections

SECTION	ROUTE	PURPOSE
Overview	<code>/dashboard/admin</code>	Summary metrics, user list, recent audit entries, audit-chain health badge
User Detail	<code>/dashboard/admin/users/[userId]</code>	Name, email, role, Stripe status, deals
Partners / API Integrations	<code>/dashboard/admin/partners</code>	API key lifecycle; deal assignment

SECTION	ROUTE	PURPOSE
Ops	/dashboard/admin/ops	Operational health: release health, webhook health, reconciliation issues, admin audit-log panel, cross-entity search
Subscriptions	/dashboard/admin/subscriptions	Subscription tier management
Design-Partner Applications	/dashboard/admin/design-partner-applications	Inbound design-partner pipeline

Audit-log and reconciliation review are served by the Overview and Ops pages plus read-only admin APIs (/api/admin/audit-log , /api/admin/reconciliation); there are no separate /dashboard/admin/audit or /dashboard/admin/reconciliation pages.

9.2 Admin Promotion Restrictions

Default state: ADMIN_PROMOTION_ENABLED=false

When false: no promote button exists in the dashboard; /api/admin/promote returns 403 for all callers; no way for any user to elevate to admin via the application.

When true (intentional, operator-enabled): caller must be admin role + MFA (AAL2) passed; self-promotion is blocked; rate limiting applies; admin justification required (X-Admin-Justification header, ≥ 20 chars); full audit log entry written + dual write to admin_audit_log .

9.3 UI Consistency Rules

Admin dashboard UI must use consistent dark background styling, uniform row heights in tables, uniform badge styles for status indicators, uniform button styles, consistent navigation (back links, breadcrumbs, section labels), no light-theme modals appearing in dark-theme context, and no inconsistent error card styles.

SECTION 10 — DEMO SYSTEM DOCTRINE

10.1 Demo Reset Safety Rules

✓ DEMO RESET MUST	✗ DEMO RESET MUST NOT
Affect only demo data (demo deal IDs/slugs)	Accept arbitrary user-provided deal IDs as targets
Be idempotent (calling twice = same clean state)	Perform broad deletes without demo-scope filtering
Require authentication (admin or demo-admin role)	Expose service-role secrets to frontend
Be disabled in production unless DEMO_RESET_ENABLED=true	Touch live production data under any circumstances

✓ DEMO RESET MUST	✗ DEMO RESET MUST NOT
Log the reset operation to the audit trail	Trust caller-provided IDs without demo scope verification
Return machine-readable success/failure response	

10.2 Demo Routes

ROUTE	PURPOSE
/demo-live	Primary demo entry; directs to guided demo
/demo-live/deal/harbor	Harbor Draw #3 — primary demo scenario (\$2.18M)
/demo-live/deal/riverside	Secondary demo scenario
/demo-live/deal/harbor-dispute	Dispute-isolation scenario
/demo-live/walkthrough	Guided walkthrough
/demo-live/admin	Demo admin view (operational visibility demo)
/demo-live/funder	Funder dashboard demo
/demo-live/contractor	Contractor demo
/demo-live/audit	Audit-trail demo
/demo-live/procore	Procore × Vektrum simulated integration concept — governed by Section 18. Fixture-driven only; noindex; no production imports

10.3 Demo Copy Rules

Approved: "This is a simulated demo. No real funds are moved." / "Authorization is demonstrated. Payment execution would flow through your selected rail."

Mandatory on /demo-live/procore (verbatim, enforced by test): "Simulated integration concept — fictional demo data only. No external system is connected and no funds are moved."

Banned: "Vektrum moves money to the contractor" / "Project Trust Account" / "Vektrum Project Trust Agreement" / specific AI provider stack names in public-facing demo copy.

SECTION 11 — COPYWRITING DOCTRINE

11.1 Approved Language

CONCEPT	APPROVED WORDING
What Vektrum is	Conditional authorization infrastructure for construction disbursements
Core capability	Release-control infrastructure; 10-condition release gate
Authorization principle	Authorization separated from execution
Payment language	Partner-controlled payment process; external/manual execution; Stripe Connect automated execution
Who governs	Vektrum governs authorization and records proof
Custody language	Vektrum does not hold funds in its own bank account or act as escrow
Audit trail	Append-only, hash-chained, tamper-evident audit log
AI pre-review	AI-assisted draw pre-review; AI informs; the gate decides
Gate promise	All required conditions must pass before a release is authorized
Trigger	Funder-triggered, system-enforced
Ecosystem posture	Keep the project system. Add release enforcement.

11.2 Banned Language and Corrections

BANNED PHRASE	APPROVED REPLACEMENT
"Vektrum moves money"	"Vektrum authorizes releases; payment flows through your selected rail"
"Vektrum executes wires"	"Partners execute wires; Vektrum records authorization and proof"
"Vektrum holds funds"	"Funds remain with your existing custody infrastructure"
"Trust account" / "Project Trust Account"	Avoid entirely; remove from all copy
"Escrow replacement"	"Vektrum integrates with your existing escrow process"
"AI approves" / "AI clears"	"AI informs; the gate decides"
"Tamper-proof"	"Tamper-evident"
"Stripe required"	"Stripe Connect is one supported execution rail"
"7-condition gate" or "8-condition gate"	"10-condition release gate"
"Authorise" / "authorisation"	"Authorize" / "authorization" (US English throughout)

11.3 How It Works — Correct Structure

The "How It Works" page must be operational, not pitch-deck focused. Steps in correct sequence:

1. Draw submitted with supporting documentation
2. Documentation reviewed; AI-assisted draw pre-review flags risk signals
3. Release gate evaluates all required conditions deterministically
4. Funder authorizes release (all conditions passed)
5. Selected rail executes: Stripe Connect automated transfer or partner-controlled external execution
6. Audit trail updated: hash-chained event recorded with proof, method, reference, and actor
7. Reconciliation state updated: billing records, deal financials, token status confirmed

11.4 Integration-Status Vocabulary (new in v1.1)

Every integration or capability claim — internal or external — must carry exactly one status, and copy must use the matching tense:

STATUS	MEANING	REQUIRED EXTERNAL WORDING
Implemented and verified	In current code, supported by passing tests or direct inspection	"The integration does..."
Implemented but not fully verified	In code, but runtime/end-to-end verification unavailable	"The current implementation is designed to..."
Simulated prototype	Demonstrated with mock/fixtures data; no production connection	"The simulated workflow demonstrates how..."
Planned	Documented future capability, not implemented	"The planned integration would..."
Unsupported	No reliable evidence	Omit from all marketing copy
Conflicting evidence	Sources disagree, unresolved	Omit until resolved

Never convert "simulated prototype," "planned," or "implemented but not fully verified" into present-tense production copy. A connected sandbox, a visible "Connect" button, a mocked screen, or a demo route alone does not prove an end-to-end production integration. Partner-facing decks must map every factual statement to a claim ledger (see [procure-submission/better-together-v2/03-claim-ledger.md](#) for the reference implementation of this practice).

SECTION 12 — INVESTOR AND MARKET DOCTRINE

12.1 How to Explain to Investors

Opening framing: "Every construction loan requires a draw process. That draw process has near-zero software investment at the enforcement layer. Draw governance still runs on email, PDFs, and the judgment of individual underwriters. Vektrum places a deterministic, auditable enforcement layer between draw approval and fund disbursement."

Market size framing: \$550B+ annual US construction starts. Every one requires a draw process. The release-control layer has seen near-zero software investment. One SKU (release governance) attaches to every disbursement = high-frequency, high-margin SaaS on top of existing loan volume.

Network effect: Each lender brings multiple deals; each deal brings contractors already proven on the platform. Compounding switching cost.

Do not claim in investor conversations: AI company status, revenue figures you cannot substantiate, signed customers when only in design-partner conversations, Stripe-only rail, or "tamper-proof" audit.

12.2 How to Explain to Construction Lenders

Opening: "You have an existing draw process. Vektrum doesn't replace any of that. It sits before your disbursement execution and verifies that all required conditions are met before any dollar moves."

Key benefits: Reduces risk of fund misapplication. Creates evidence for LP reporting and regulatory compliance. Mechanic's lien protection via lien waiver enforcement. Reduces human judgment load on individual underwriters.

12.3 How to Explain to Title Companies

Opening: "Vektrum is not a title replacement. Your title company process stays exactly as it is. Vektrum is the layer the lender uses to verify release conditions are met before they authorize you to release funds."

Integration story: You receive a release authorization signal from Vektrum. You disburse through your existing infrastructure. You confirm back through the API. Both parties share a permanent audit record.

12.4 How to Explain to Contractors

Opening: "Vektrum shows you exactly what conditions are required for each draw to release and the current status of each condition. If your lien waiver is missing, you'll see it. If there's an open change order, you'll see it. No mystery about why a draw is delayed."

12.5 How to Explain to Technical Reviewers

- Next.js 15 application; Supabase (PostgreSQL + RLS) for persistent storage
- Row-Level Security enabled on all 31 tables (verified against migrations 2026-07-27)
- Hash-chained, append-only audit log with SHA-256 row and chain hashes (`hash_schema_version` 2; `token_hash` + `partner_ack_hash` binding)

- ed25519-signed authorization tokens per release (graceful fallback to unsigned); status flow issued → delivered → confirmed/failed/expired/revoked; TTL 24 h (Stripe rail) / 30 days (external rail); amounts locked at issuance
 - Rate limiting via database RPC (`check_rate_limit`); `financial_write` and `admin_write` fail closed
 - Stripe Connect for automated rail; Partner API for institutional rail, with optional HMAC-signed outbound `release.authorized` webhooks
 - AI draw review via sonar-pro (primary) → claude-sonnet (fallback) → gpt-4o (fallback); 48 h freshness window
 - 117 automated test files across release gate (55 checks), security, webhook idempotency, admin safety, demo safety — plus the Procore suites: `test:procore` (OAuth security, Phase 1A route/RLS safety, funder access) and `demo-procore-prototype` (70 behavioral + banned-language checks)
 - Migration-based schema management; immutability triggers on `authorization_tokens` and `audit_log`
 - Procore connectivity (Phase 1A): sandbox-only OAuth, read-only client, AES-256-GCM token storage, server-only RLS — see Section 18
-

SECTION 13 — ENGINEERING DOCTRINE

13.1 Before Any Edit

1. Read `CLAUDE.md` at the repo root
2. Read `docs/ai/MASTER_CONTEXT.md` — current architecture and non-negotiables
3. Read `docs/ai/BACKLOG.md` — current priorities and what not to touch
4. Identify exactly which files are relevant — do not load unnecessary context
5. Produce a plan describing what will change and why
6. For CRITICAL or HIGH risk areas, wait for explicit approval before executing

13.2 After Any Edit

1. Run `npm run build` — document any new errors vs pre-existing
2. Run `npx tsc --noEmit` — zero new TypeScript errors
3. Run relevant tests: `npx tsx tests/[specific].test.ts`
4. Run `npm test` for full suite if shared infrastructure changed
5. Document exactly which files changed and why
6. Provide manual test steps for QA if UI changed
7. Confirm no secrets, real API keys, or real customer data are in the changeset

13.3 Test Expectations (TDD Required)

1. Write failing test describing expected behavior
2. Run test — confirm it fails for the right reason
3. Implement the smallest change that makes the test pass
4. Run all tests — confirm no regressions
5. Refactor if needed — tests protect behavior

13.4 Safe vs Dangerous Areas

AREA	RISK LEVEL	RULE
Release gate logic	CRITICAL	Do not modify without test coverage and explicit approval
Authorization token issuer	CRITICAL	Do not modify without understanding B3 hardening implications
<code>reserve_release_funds()</code> RPC	CRITICAL	Do not change behavior; double-spend protection
Audit chain hashing	HIGH	Do not change algorithm or payload structure without versioning
Admin promotion logic	HIGH	Default off; MFA required; self-promotion blocked
Billing calculation	HIGH	Do not change <code>calculateFee()</code> without test coverage
RLS policies	HIGH	Do not disable or weaken; test all changes
Partner API authentication	HIGH	Hash comparison; do not expose raw keys
Procore integration config/safety invariants	HIGH	Sandbox-only + read-only + encrypted tokens; see Section 18.2
Public copy	MEDIUM	Requires copy truth-lock review per Section 11
Demo reset	MEDIUM	Demo scope filter required
UI components (non-financial)	LOW–MEDIUM	Follow approved language
Test files	LOW	Adding tests is always encouraged

13.5 Rails Adapter Pattern — Do Not Bypass

```
// CORRECT: Use rail adapter
const adapter = getRailAdapter(railScope)
const dispatchResult = await adapter.dispatch({ ... })

// INCORRECT: Inline Stripe call in route – NOT ALLOWED
const transfer = await stripe.transfers.create({ ... })
```

Known deviation (flagged in the 2026-07-27 conformance audit, unresolved):

`src/app/api/deals/[dealId]/retainage/release/route.ts` calls `stripe.transfers.create` inline instead of going through the rail adapter. The rule stands; the deviation is tracked in Section 17.2 and must be refactored — with tests and explicit approval — rather than used as precedent.

13.6 Authorization Token Issuance — Required Fields (B3 Hardening)

```
await issueAuthorizationToken({
  milestoneId,
  dealId,
  payeeId,
  funderId,
  railScope,
  netToContractor,
  grossAmount,
  feeAmount,           // B3 hardening – REQUIRED. Do not omit.
  idempotencyKey,
  issuedBy,
  graphCommitment,    // Tier D – if evidence graph is built
})
```

SECTION 14 — RELEASE CHECKLIST

14.1 Build and Test

- `[] npm run build` passes with no new errors
- `[] npx tsc --noEmit` — zero new TypeScript errors
- `[] npm test` — all tests pass or pre-existing failures are documented
- `[]` New tests written for all behavior changes (TDD)
- `[]` No new `console.error` or unhandled promise rejections in build output

14.2 Security

- ☐ No real API keys, secrets, or tokens in the changeset
- ☐ No `SUPABASE_SERVICE_ROLE_KEY` in `NEXT_PUBLIC_*` variables
- ☐ Admin promotion remains disabled by default
- ☐ Admins cannot trigger release via any new routes
- ☐ Partners cannot bypass the gate via any new routes
- ☐ New financial-write routes have rate limiting applied
- ☐ New admin routes require MFA
- ☐ RLS is still enabled on all tables touched by migrations
- ☐ Stripe webhook handling includes HMAC verification and event deduplication
- ☐ Procore invariants intact: sandbox-only environment check, `PROCORE_WRITE_OPERATIONS_ENABLED=false`, read-only client, encrypted token storage, funder-only routes (`npm run test:procore` passes)

14.3 Copy and Legal-Risk

- ☐ No "Vektrum moves money" or equivalent
- ☐ No "Vektrum holds funds" or equivalent
- ☐ No "escrow replacement" or "trust account" or "Project Trust Account"
- ☐ No "AI approves" or "AI decides"
- ☐ No "tamper-proof" — only "tamper-evident"
- ☐ No "Stripe required" — external rail messaging present where relevant
- ☐ Gate described as "10-condition" — not 7 or 8
- ☐ Partner docs do not overclaim outbound webhooks if unimplemented
- ☐ Demo copy does not claim real fund movement
- ☐ Integration claims carry their true status per Section 11.4 (no simulated/planned/sandbox capability described in present-tense production language; no "live Procore integration," "Marketplace app," or Procore endorsement claims)

14.4 Partner API

- ☐ No new partner endpoints that bypass the release gate
 - ☐ Partner authentication remains hash-based
 - ☐ Partner scope isolation enforced (cross-partner access returns 403)
 - ☐ Confirm and fail endpoints are idempotent
 - ☐ Audit rows bind `partner_ack_hash` and `token_hash`
 - ☐ Postman collection uses placeholder keys only
-

SECTION 15 — AI AGENT BEHAVIOR RULES

15.1 Reasoning Order

1. Read this document first. No work begins without it.
2. Identify the task category: code, copy, docs, security, or architecture.
3. Check this document for governing rules on that category.
4. Identify the minimum required change that achieves the goal.
5. Check for dangerous areas (Section 13.4). If CRITICAL or HIGH, state this explicitly.
6. Produce a plan before executing. For CRITICAL areas, wait for explicit approval.
7. Execute with precision. No scope creep. State any "while I'm here" changes explicitly.
8. Verify with tests and build. Do not claim success without running the checks.
9. Report honestly: if tests fail, say so; if something was skipped, say so.

15.2 When to Stop and Ask

The AI must stop and ask for explicit approval before proceeding when the task involves:

- Any change to the release gate logic
- Any change to billing or financial calculations
- Any change to admin security boundaries
- Any change to audit log behavior or hashing scheme
- Any copy change that makes custody, escrow, or AI approval claims
- Any new route that could create a path for non-funder actors to trigger release
- Any test fixture that might touch production data
- Any change that would enable Procore write operations, point the Procore integration at a non-sandbox environment, weaken Procore token encryption/RLS, or present the Procore relationship as live/production in any copy

15.3 Required Change Summary Format

```
## Change Summary
Files modified:
- [file path]: [one-sentence description of what changed and why]

Tests added or modified:
- [test file]: [what behavior is now tested]

Build/test status:
- npm run build: [pass/fail]
- npx tsc --noEmit: [pass/fail / N new errors]
- npm test: [pass/fail] [count of failing tests if any]
- Pre-existing errors: [list if any]

Security/custody/gate risk: [low / medium / high] – [reason]
Copy/legal risk: [low / medium / high] – [reason]

Manual test steps:
- [step 1]
- [step 2]

Next recommended action: [specific concrete next step]
```

15.4 What the AI Must Never Do Unilaterally

Without explicit approval, the AI must never:

- Change the 10 release conditions or their labels
 - Change the billing calculation formula
 - Change admin role permissions
 - Change partner scope isolation logic
 - Disable rate limiting on any route
 - Weaken RLS policies
 - Add any pathway for admins to release funds
 - Change the authorization token hashing scheme
 - Change the audit chain hash algorithm or payload structure
 - Remove MFA requirements from any route
 - Add public copy that makes custody, escrow, or AI approval claims
 - Commit real API keys, secrets, or tokens
 - Enable Procore write operations, non-sandbox Procore environments, or unverified OAuth scopes; or publish copy claiming a live Procore integration, Marketplace listing, or Procore endorsement
-

SECTION 16 — EXAMPLE RESPONSES AND PATTERNS

16.1 To a Title Company

"Not at all. Vektrum is the governance layer that your lender client uses to verify that all release conditions are met before they authorize you to release funds. Your disbursement process stays exactly as it is. We integrate with your process through our Partner API — when you release funds on behalf of a lender using Vektrum, you confirm the execution through our API and both parties get a shared audit record of what was authorized, when, and on what basis."

16.2 To a Construction Lender

"Vektrum sits before your title and escrow process, not instead of it. When a draw is submitted, Vektrum evaluates all required conditions — lien waivers, change orders, contract status, funding availability, contractor onboarding, and AI-assisted draw review. If everything passes, you authorize the release and Vektrum records that authorization with a full audit trail. Your LP reporting now has a machine-readable trail of every release: what was authorized, why, by whom, and what documentation was on file."

16.3 To an Investor

"A general payments company solves 'how do we move money faster.' Vektrum solves 'how do we verify that money is allowed to move at all, and how do we prove it?' That's a fundamentally different problem. And because Vektrum works with whatever payment rail the lender already uses — Stripe Connect for automated releases, or their existing title/escrow/treasury process — there's no switching cost. We're not replacing the payment rail. We're enforcing whether the payment should happen at all."

16.4 To Procore or a Procore Customer (new in v1.1)

"Procore is the system of record for the project — schedules, documents, cost management. Vektrum doesn't touch any of that. Vektrum is the release-governance layer the lender or owner uses before construction funds are disbursed: a deterministic 10-condition gate, funder authorization, and a tamper-evident audit trail. We've built a simulated workflow that demonstrates how Procore-originated records would feed that review, and our current sandbox connectivity is read-only against the Procore Developer Sandbox. Keep the project system; add release enforcement."

16.5 Correcting Banned Wording

DRAFT (BANNED)	CORRECTED (APPROVED)
"Vektrum's AI clears your draws for payment"	"Vektrum's AI-assisted draw pre-review flags missing documentation, conflicts, and risk signals. The deterministic gate then evaluates all required conditions. AI informs; the gate decides."
"Funds are held in Vektrum's trust account"	"Funds remain with your existing custody infrastructure; Vektrum enforces release conditions before authorizing disbursement"
"Tamper-proof audit trail"	"Tamper-evident, hash-chained, append-only audit trail"

DRAFT (BANNED)	CORRECTED (APPROVED)
"8 required conditions"	"10 required conditions"
"Escrow replacement for modern lenders"	"Release governance for construction lenders, integrating with your existing title, escrow, and treasury processes"
"Our AI approves draws in seconds"	"AI-assisted draw review runs before the deterministic gate — flagging missing documents, conflicts, and risk signals"
"Our live Procore integration syncs your projects in real time"	"The current implementation is designed to connect to the Procore Developer Sandbox (read-only). The simulated workflow demonstrates how Procore-originated records would feed a lender's draw review. Production integration is planned."

SECTION 17 — CURRENT KNOWN BACKLOG (July 2026)

17.1 Critical / Before Merge

ITEM	STATUS	ACTION REQUIRED
<code>/auth/logout</code> 404	Needs verification	Verify logout route exists; fix if returning 404
Dashboard user navigation	Partly addressed	Route <code>/dashboard/admin/users/[userId]</code> exists (verified 2026-07-27); runtime behavior with real data still needs manual QA
Partner API docs webhook claims	Resolved 2026-07-27	Outbound <code>release.authorized</code> webhooks are implemented and tested; docs are compliant with §8.5

17.2 High / Before Pilot

ITEM	ACCEPTANCE CRITERIA
Route/navigation smoke tests	All critical routes verified non-404 via test or checklist
Admin dashboard full verification	No promote button; admin promotion disabled; partners section works; no raw secrets exposed
Demo reset full verification	Demo-scope only; idempotent; no production data touched; clean reload after reset (known issue: demo contractor reset may not restore all buttons/state)
Retainage release rail-adaptor refactor	<code>retainage/release</code> route stops calling <code>stripe.transfers.create</code> inline and dispatches through <code>getRailAdapter</code> (§13.5 known deviation); behavior protected by tests before and after

17.3 Procore Integration Track (new in v1.1)

Current truth (verified 2026-07-24; full evidence in `procore-submission/better-together-v2/`):

LAYER	STATUS
Simulated end-to-end joint workflow at <code>/demo-live/procore</code> (fictional Harbor Point \$2.18M Draw #07)	Simulated prototype — 70 behavioral checks passing
Sandbox OAuth + read-only identity/project-list client (Phase 1A)	Implemented but not fully verified — <code>test:procore</code> suites pass; no runtime E2E evidence recorded
Procore data sync, write-back, webhooks, production connectivity	Planned — no code; writes hard-disabled
Marketplace listing, Procore endorsement, joint customers/metrics	Unsupported — do not claim

Next steps, in order: 1. Record one end-to-end Developer Sandbox run (connect → identity → project list) to upgrade Phase 1A to verified. 2. Implement and validate the first read path (one Procore object family into a draw review) against official API docs and permissions. 3. Run one design-partner pilot on a lender-funded Procore project to produce the first labeled joint metrics. 4. Complete Procore technical review; pursue Marketplace readiness. 5. Add documented `PROCORE_*` placeholders to `.env.example` (names only, no values).

17.4 Test Priorities

1. Release gate tests — all 10 conditions + bypass attempts
2. Partner API confirm/fail tests — idempotency, scoping, audit binding
3. Admin promote disabled tests — verify default-off behavior
4. Demo reset safety tests — verify demo-scope isolation
5. Route existence/navigation smoke tests
6. Stripe webhook idempotency tests
7. RLS/DB trigger expectation tests
8. Procore safety tests — keep `test:procore` + `demo-procore-prototype` green on any Procore-adjacent change

17.5 Do Not Touch Without Explicit Instruction

- Release gate logic (`src/lib/engine/release-gate.ts`)
- Billing calculation formula (`src/lib/engine/billing.ts`)
- Stripe webhook transaction logic
- Admin permission boundaries
- Partner API request/response shapes
- Audit chain hashing scheme
- RLS policies

- Authorization token immutability triggers
 - Public site copy, unless product behavior has changed
 - Procore config safety invariants (`src/lib/integrations/procore/config.ts`)
-

SECTION 18 — ECOSYSTEM INTEGRATION DOCTRINE: PROCORE (new in v1.1)

18.1 Relationship Truth

The Vektrum $\rightleftharpoons$ Procore relationship is, today, a **simulated integration concept with early sandbox groundwork**. It is not a live integration, not a partnership, not a Marketplace listing, and not an endorsement. Every artifact — code, copy, decks, demos — must reflect this until the underlying status changes and is verified.

Canonical positioning (approved): **"Procore keeps the project record. Vektrum adds enforceable release governance before construction funds are disbursed."** Vektrum is complementary to Procore — never a competitor, never a replacement, and never diminished-Procore framing ("Procore lacks...", "Vektrum fixes Procore") in any direction.

18.2 Phase 1A Technical Invariants (enforced in code and tests — do not weaken)

1. **Sandbox-only.** `getProcoreConfig()` hard-fails unless `PROCORE_ENVIRONMENT=sandbox` ; authorize/token/API URLs are pinned to the Procore Developer Sandbox hosts. No production Procore environment may be configured without explicit product + security approval.
2. **Read-only.** `PROCORE_WRITE_OPERATIONS_ENABLED` must be `false` ; the client makes no POST/PUT/PATCH/DELETE calls (test-enforced). Write-back is a planned capability requiring its own review.
3. **Funder-gated.** All `/api/integrations/procore/*` routes require the funder role server-side. Contractors and admins have no Procore surface.
4. **Token security.** OAuth state stored only as SHA-256 hash, single-use, 10-minute expiry. Access/refresh tokens stored only AES-256-GCM-encrypted (`PROCORE_TOKEN_ENCRYPTION_KEY` , 32-byte, versioned). `procore_oauth_transactions` and `procore_connections` have RLS enabled with deliberately **no** authenticated policies — server-only access; sanitized status via server route. No raw tokens, secrets, or ciphertext in any UI, log, or response.
5. **Scope discipline.** No OAuth scope is sent unless explicitly configured (`PROCORE_SCOPES`). Redirect URI allowlisted to localhost:3000 or vektrum.io callback only.
6. **Kill switch.** `PROCORE_INTEGRATION_ENABLED=false` disables the integration fail-closed.
7. **Audit.** Connect, callback success/failure (with stage), disconnect, and project selection are audit-logged.

8. **Blast-radius rule.** The Procore connection stores connection and selected-project metadata only. No deal, release, payment, SOV, or financial data flows from Procore into Vektrum in Phase 1A.

Regression gates: `npm run test:procore` and `npx tsx tests/demo-procore-prototype.test.ts` must pass on any Procore-adjacent change.

18.3 Simulated Prototype Rules (`/demo-live/procore`)

- Pure fixture-driven state machine. No fetch, no `process.env`, no OAuth, no crypto, no imports of production engine/auth/Supabase/Stripe code (test-enforced).
- All data fictional; identifiers inert (`DEMO-` / `MOCK-` / `SIM-` prefixes — e.g., `DEMO-AUTH-HPMC-07`, `DEMO-MWTE-88241`).
- The disclosure line (Section 10.3) must remain verbatim in source.
- The demo must preserve product truth mechanically: pre-review before gate; conditions 7 and 10 block until resolved at source; condition 4 not-applicable on the external rail (never "passed"); authorization does not confirm execution; external confirmation only after authorization; reset restores exact initial state.
- Procore object mappings shown in the prototype are **conceptual** — labeled "API feasibility and permissions not yet validated." Do not invent or present Procore API object mappings as validated anywhere.

18.4 Procore-Facing Language

✓ APPROVED	✗ BANNED (UNTIL VERIFIED TRUE)
"Procore keeps the project record; Vektrum adds release governance"	"Live Procore integration" / "fully integrated" / "seamless"
"The simulated workflow demonstrates how Procore-originated records would feed the lender's draw review"	"Real-time sync" / "one-click payment from Procore"
"The current implementation is designed to connect to the Procore Developer Sandbox — read-only, write operations disabled"	"Available in the Procore Marketplace" (no listing exists)
"The planned integration would return authorization proof to the project record"	"Procore-approved" / "Procore partner" / any endorsement claim
"Keep the project system. Add release enforcement."	"Procore tracks; Vektrum pays" / "Vektrum is Procore for payments"
"Vektrum complements Procore, Procore Pay, and existing waiver tools"	"Better than Procore Pay" / "Procore lacks payment controls"

Slide 5-class rules for any joint material: no fabricated customers, quotes, metrics, or Procore endorsement; illustrative scenarios (the \$2.18M simulated draw; the published \$15K/\$9M dispute-isolation example) must always carry an "illustrative — not customer data" label.

18.5 Partner-Material Process

Any Procore-facing deck, brief, or doc must be built the way `procore-submission/better-together-v2/` was built: repository audit first → integration truth table → claim ledger (every statement mapped, Tier D excluded) → copy in status-correct tense → visual QA. Accuracy outranks excitement; a prototype is never quietly converted into a live integration.

SECTION 19 — FINAL NON-NEGOTIABLES

These are the most important rules in this document. They must be honored in every piece of work, every AI output, every line of code, and every word of copy. There are no exceptions.

1. **Vektrum does not hold funds in its own bank account or act as escrow.** Funds are held by Stripe Connect (for Stripe rail deals) or by the funder's existing title company, escrow company, bank, or treasury infrastructure. Any copy, code, or AI output that implies otherwise must be corrected.
2. **Authorization is separated from execution.** Vektrum authorizes whether a release is allowed. Payment execution happens through the selected rail — not through Vektrum. The rail adapter exists to enforce this separation in the codebase.
3. **Admins cannot release funds.** The admin role is operational. No dashboard feature, no API endpoint, no code path should give admin users the ability to trigger a construction release. Any such path is a critical security bug.
4. **Partners cannot bypass the release gate.** Partner API endpoints confirm or fail external execution after authorization. They do not create authorization, and they do not skip the gate. Any partner endpoint that creates a release without gate evaluation is a critical bug.
5. **AI informs; the deterministic gate decides.** No work should result in copy, code, or documentation that implies AI makes the release decision. The AI pre-review runs before the gate. The gate — all 10 conditions — decides.
6. **Do not overclaim webhooks, custody, automation, or payment execution.** Do not document outbound webhooks as live unless the implementation exists. Do not claim Stripe is required. Do not claim Vektrum automates all payments. Do not claim Vektrum is an escrow or custody provider.
7. **Public copy must stay rail-neutral and legally careful.** The public site must not imply Stripe is the only rail, use trust account language, imply Vektrum holds funds, use "tamper-proof," describe AI as approving payments, or claim a gate condition count other than 10.
8. **The 10-condition gate is fixed and public.** The gate has exactly 10 conditions. They must not be described as 7, 8, or 9. They must not be reordered, removed, or combined without explicit product approval. Any test, copy, or documentation describing fewer conditions is incorrect.
9. **Raw API keys are shown once. Never again.** The Partner API key is generated, shown once at creation, and stored only as a SHA-256 hash. If a partner loses their key, it must be rotated. Any UI that shows a raw key after initial creation is a security bug.
10. **No service-role secrets in the frontend. Ever.** `SUPABASE_SERVICE_ROLE_KEY` , `STRIPE_SECRET_KEY` , `VEKTRUM_TOKEN_SIGNING_KEY_PRIVATE` , `PROCORE_CLIENT_SECRET` , `PROCORE_TOKEN_ENCRYPTION_KEY` ,

and all equivalent server-side secrets must never appear in `NEXT_PUBLIC_*` environment variables, client components, or any code bundled into the browser.

11. **Integration claims must match integration reality.** Every ecosystem capability — Procore or future integrations — is described with its true status per Section 11.4: implemented and verified, implemented but not fully verified, simulated prototype, planned, or unsupported. Simulated and planned capabilities are never written in present-tense production language, and no Marketplace, certification, partnership, or endorsement claim is made without documented proof.

Vektrum Operating Constitution · Version 1.2 · July 2026 · code-conformance audited 2026-07-27 This document is the permanent operating law for all work on Vektrum. When in doubt: be conservative, be precise, and ask. vektrum.io